B11 Machine learning workflow

Splits, Pipeline, ColumnTransformer, cross-validation and hyperparameter search.

B11-L1 Train, validation and test, split by the right unit

Tune on validation, measure once on test, and split by the unit that will be new in production.

```python
from sklearn.model_selection import GroupKFold, GroupShuffleSplit

splitter = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=7)
train_idx, test_idx = next(splitter.split(df, groups=df["respondent_id"]))
```

Roles of train, validation and test	Fit, choose, measure once.
Rows repeat per respondent	Split by group (GroupKFold, GroupShuffleSplit), or the model learns people.
Group leakage demonstration	Random CV 0.995, GroupKFold 0.468 on a label unrelated to the features.
Rare class	stratify=y keeps the class share in each set.

B11-L2 Pipeline and ColumnTransformer

Put every learning step in one Pipeline, split columns with a ColumnTransformer, and deploy the fitted object itself.

```python
prep = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()),
  NUMERIC),
    ("cat", make_pipeline(SimpleImputer(strategy="most_frequent"),
                          OneHotEncoder(handle_unknown="ignore")), CATEGORICAL),
])
model = Pipeline([("prep", prep), ("clf", LogisticRegression(max_iter=1000))])
```

Why Pipeline	Every learning step refits per CV fold, and one fitted object goes to production.
OneHotEncoder default for unseen categories	handle_unknown="error": raises ValueError. Use "ignore" for production.
ColumnTransformer default for unlisted columns	remainder="drop": silently dropped.
Feature names after preprocessing	get_feature_names_out(), prefixed with step names such as num__age.

B11-L3 Cross-validation and honest hyperparameter search

Report CV mean and spread, tune with the reported metric, and estimate performance with nested CV or an untouched test set.

```python
res = cross_validate(pipe, X, y, cv=StratifiedKFold(5, shuffle=True, random_state=7),
                     scoring=["roc_auc", "average_precision"])
res["test_roc_auc"].mean(), res["test_roc_auc"].std()
```

```python
search = GridSearchCV(pipe, grid, cv=inner, scoring="roc_auc")
honest = cross_val_score(search, X, y, cv=outer, scoring="roc_auc")
```

Report CV results as	Mean and spread across folds, for the metric you care about.

Pipeline parameter names in a grid	step__param, such as clf__C. make_pipeline names steps after their class.
Is best_score_ an honest estimate?	No. Noise simulation: 0.598 best_score_ against 0.515 nested. Use nested CV or a held-out test set.
Nested CV in scikit-learn	cross_val_score(GridSearchCV(pipe, grid, cv=inner), X, y, cv=outer).